check_cm_export_charset_valid

check_cm_export_charset_valid.sh — CM-EXPORT-CHARSET-VALID

Revision: 3 **Last modified:** 2026-08-26T13:05:00Z **Authority:** §11.4.238 (automated QA is the discoverer) · §11.4.135 (monotone ratchet) · §11.4.249 (role separation) · §11.4.18 (script documentation) **Scope:** pre-build invariant 50 and the two scripts that surround it

Overview

Every generated `.html` export must declare its own character encoding. This gate counts the ones that do not, and compares that count to a **persisted ratchet baseline**.

It exists because of a measured coverage escape. During BOB-169 the export regime asserted *presence* and *mtime* but never *validity*: 301 of 334 generated exports were charset-less fragments, the PDFs rendered from them carried 12,629 corrupted lines across 281 files, and every gate stayed green. An agent reading a PDF found it — not the regime. Under §11.4.238 that out-of-band discovery **is** the defect; closing the corruption without closing the detection gap would have been the violation.

The two invariants

This gate asserts two things, and either one can refuse:

1. **The corpus** — the charset-less count does not *exceed* the baseline.
2. **The ratchet** — the baseline is *current*. A count **below** the baseline means the corpus healed and the bar never followed it down, so the gate would be silently licensing a regression back to the old

count. That staleness is itself a finding (BOB-182), and it is refused rather than mentioned in passing prose that a human may never read.

Adoption model — operator decision

Recorded 2026-08-26 under §11.4.66 / §11.4.224(E), verbatim: **“Keep the monotone ratchet.”** The count may only go down, never up.

An immediate hard floor was offered and **not** chosen — 301 pre-existing violations would have made the build unreachable, which §11.4.234 forbids. Per-corpus phase-in and changed-code-only-with-deadline were also offered and not chosen. This is consumer DATA per §11.4.35: the operator’s stated choice, not an agent’s invented default. It is recorded here so a future reader does not re-litigate a settled call (§11.4.112(5)).

The three components and their roles (§11.4.249)

The threshold is data, not code, and the component that *enforces* it is not the component that *writes* it. A gate that wrote its own threshold during a pre-build run would be a PRODUCER as well as a GATE — the collapse §11.4.240(C) names as the one that yields weakened thresholds.

File	§11.4.249 role	Writes the baseline?
`scripts/pre_build/lib/cm_export_charset_scan.py`	**ORACLE** — counts violations; knows nothing about thresholds	no
`scripts/pre_build/lib/cm_export_charset_baseline_read.sh`	**READER** — the single validator of what a baseline may be	**no — read-only by construction**

File	§11.4.249 role	Writes the baseline?
`scripts/pre_build/check_cm_export_charset_valid.sh`	**GATE** — compares the count to the persisted baseline	**no — contains no write path at all**
`scripts/pre_build/tighten_cm_export_charset_baseline.sh`	**PRODUCER** — explicitly invoked, never on the pre-build path	**yes, and only downward**
`scripts/pre_build/cm_export_charset_valid.baseline`	the threshold as **DATA** (§11.4.35)	—
`tests/pre_build/test_cm_export_charset_valid.sh`	**VERIFIER** — §1.1 paired mutations, outside the gate	no

The gate and the tightener share **one** oracle *and* **one** baseline reader, so neither the number they act on nor the rule for what counts as a valid threshold can diverge between them. The reader exists because the round-1 BLOCKING was exactly that: one parse rule written twice and fixed once.

The separation is a **capability**, not an instruction (§11.4.240(B)): the gate source has no line that could write the baseline, and the test suite asserts this both structurally (the source contains no write to `BASELINE_FILE`) and at runtime (the baseline file's sha256 is unchanged across a below-baseline gate run, which is precisely the moment a self-writing gate would rewrite its own bar).

Honest boundary

The oracle and the gate are adjacent, not remote — the gate invokes the scan directly. That is acceptable for the precise reason §11.4.249 gives: the forbidden oracle=gate collapse is an oracle that *cannot say "cannot decide"* and therefore defaults to **allow**. Every cannot-decide branch here fails **closed** (§11.4.252): a blind enumeration, a detector that cannot be shown to discriminate, and a missing or malformed baseline all refuse.

Loosening is a tracked diff, never ambient

There is deliberately **no environment variable that injects a baseline value**. Raising the bar requires editing a git-tracked file, which a reviewer sees (§11.4.142).

The previous `BOBA_EXPORT_CHARSET_BASELINE` override let any caller pass a broken corpus while leaving no reviewable trace. Measured 2026-08-26 against the pre-fix gate: `BOBA_EXPORT_CHARSET_BASELINE=9999` turned a planted violation into a `PASS`. That channel is closed, and the suite probes both shapes of the exploit — an absurd injected value and one set exactly equal to the live count.

What that guarantee rests on. The scripts' refusals raise the *cost* of a raise; they do not make one impossible. Delete-then-`--adopt`, or a hand-edited number, both reach a higher bar. What stops those is that each leaves a diff in a **tracked** file. So the guarantee exists **only once `cm_export_charset_valid.baseline` is committed** — while it is untracked, every loosening route is traceless and the defence is void. The tightener says exactly this on every successful write ("an untracked ratchet binds nothing"), and it is the literal state of the file until the commit lands.

Usage

```
# The gate. No arguments: scans the project root. Wired as pre-
        build invariant 50.
bash scripts/pre_build/check_cm_export_charset_valid.sh
        [SCAN_ROOT]

# The producer. Explicitly invoked; never runs during a build.
bash scripts/pre_build/tighten_cm_export_charset_baseline.sh
        [SCAN_ROOT]
bash scripts/pre_build/tighten_cm_export_charset_baseline.sh --
        adopt [SCAN_ROOT]
```

```
# The verifier — §1.1 paired mutations.
bash tests/pre_build/test_cm_export_charset_valid.sh
```

Gate exit codes

Exit	Meaning
`0`	count is at the baseline; no regression and the ratchet is current
`1`	a regression, a blind scan, a non-discriminating detector, or a missing/malformed baseline
`3`	**stale ratchet** — the corpus improved and the bar did not follow; run the tightener

These codes are a **contract**, not incidental non-zeros: invariant 50 branches on `{0,1,3}` to name the cause, so collapsing `3` into `1` would make it report "a NEW charset-less export landed" for a corpus that is merely un-ratcheted. The paired mutations assert each code exactly, so that collapse cannot land silently.

Tightener exit codes

Exit	Meaning
`0`	baseline lowered, or already current (no-op)
`1`	refused — would raise the bar, blind scan, or wrong mode for the file's existence
`2`	usage error

What counts as an export, and as compliant

- **A generated export** is an `.html` with a sibling `.md`. Hand-authored page furniture has no `.md` twin and is out of scope.
- **Compliant** means a `<meta ... charset...>` **element**. Structure, never the bare substring — a document is not self-describing because its prose contains the word "charset". Measured during

BOB-169: a naive `grep -qi charset` *passed* against the broken generator by matching a heading slug (§11.4.201(7)(a)).

Scanned scope

`docs/`, `scripts/`, and the repository root — 340 pairs as of 2026-08-26. Deliberately excluded: `.claude/` (agent and plugin furniture), `constitution/` and `submodules/` (separate repositories with their own governance — policing a submodule's corpus from here would violate §11.4.28 decoupling). This scope is currently implicit in the oracle's walk rather than declared in a checked-in map; see "Known gaps".

Edge cases

- **Empty corpus** → refuses. A blind scan and a perfect corpus return the same quiet zero, so a nonzero *compliant* count is required before any zero is believed (§11.4.201(6) / (7)(b) control needle).
- **No compliant file at all** → refuses; the detector cannot be shown to discriminate.
- **Baseline missing** → refuses, and names `--adopt`. The threshold is an input to the gate's correctness; a gate that invents one is asserting a condition it never checked (§11.4.252).
- **Baseline malformed** → refuses. An unresolvable signal takes the conservative-safe default (§11.4.201(4)).
- **`--adopt` when a baseline already exists** → refuses. This blocks the careless raise, **not** the determined one: deleting the file and re-adopting still reaches a higher bar. The real defence is the tracked diff, not this refusal — see "Loosening is a tracked diff" below for what that does and does not guarantee.
- **Leading-zero baseline (`08`, `010`, `007`)** → refuses as malformed. Bash reads these as octal and *errors*; `if` swallows the error as false, so before this was fixed both comparisons fell through and the gate reached `PASS` with violations present (measured 2026-08-26: `baseline=08` + 3 violations → `exit 0`, and invariant 50 swallows stderr, so the build went green). The parse is now strict: `^baseline=(0|[1-9][0-9]*)$`.
- **Baseline path is a directory** → the gate fails closed, and the tightener refuses rather than reporting a write it did not perform (`mv` would otherwise succeed by depositing the temp file *inside* the directory).
- **Baseline over 18 digits (e.g. $2^{64}+3$ = `18446744073709551619`)** → refuses. The no-leading-zero rule alone accepts it, and bash

arithmetic then wraps mod 2^{64}: measured 2026-08-26, that value with **3 violations present** returned `exit 0`, "at baseline". The sibling $2^{64}+1$ refused but printed a resolved-evidence line that was false. The accepted range is now `0`, or 1–18 digits.
- **More than one `baseline=` line** → refuses as ambiguous. `sed …p | tail -1` silently resolved such a file to its last *matching* line, so `baseline=3` followed by `baseline=08` — plausibly an operator's intended correction — resolved to 3 and passed; two conflicting well-formed lines resolved just as quietly. Ambiguity is not a value (§11.4.252).
- **Baseline path is a symlink** → refuses. `-f` follows symlinks, so a link to a target outside the tree reads normally and every later raise edits that target with no diff at all.

Known gaps

- The scanned scope above is implicit in the oracle's directory walk rather than declared as a checked-in topology map with per-exclusion justification, which is what §11.4.135 asks of an exemption set. Not addressed here (out of BOB-182's scope); worth a tracked item.
- This guide covers all three scripts; the producer additionally has its own per-script guide at `docs/scripts/tighten_cm_export_charset_baseline.md`.
- This document's `.html` / `.pdf` / `.docx` twins are not generated here. The export pipeline is owned elsewhere in this round (§11.4.119), and hand-forging a file that claims to be a generated export would be worse than an honest gap — particularly for a document about export validity.

Related

- `scripts/pre_build_verification.sh` — invariant 50 invokes the gate
- `docs/scripts/tighten_cm_export_charset_baseline.md` — the producer's guide
- `docs/QA_DISCOVERY_LEDGER.md` — the §11.4.238 coverage-escape entry for BOB-169
- BOB-169 (the escape and the original gate), BOB-182 (the ratchet that did not ratchet)